

# TRAVELAPP CONSOLIDATED ARCHITECTURE BLUEPRINT v3.0

ALFIAN TOUR - Sistem Informasi Perjalanan Umrah, Haji, Wisata dan Tour

Platform: .NET 10 + Next.js 14 | SQL Server | EF Core | Clean Architecture

# DAFTAR ISI

PART A - FRONTEND: Mobile Marketplace UI Specification

PART B - BACKEND: Alfian Tour Full Stack Blueprint v2.0

PART C - MASTER DATA ARCHITECTURE v3.0

PART D - FINANCIAL and COSTING ARCHITECTURE

PART E - SCALABILITY AUDIT and BOTTLENECK PREVENTION (NEW)

PART F - CONSOLIDATED DATABASE SCHEMA (All Tables)

PART G - COMPLETE API ENDPOINT REFERENCE

PART H - BUILD ROADMAP and IMPLEMENTATION PRIORITIES

# PART A - FRONTEND: MOBILE MARKETPLACE UI SPECIFICATION

Sumber: Prompt.txt - Next.js App Router + Tailwind CSS + shadcn/ui

## A.1 Main Concept and Design System

Premium modern mobile-first marketplace web application UI. Inspired by Shopee mobile UX but cleaner, more premium, and modern. Must feel like a native mobile marketplace app.

### Key Design Principles

[x] Mobile-first experience - on desktop always display mobile layout centered (max 430px)

[x] UI must feel fast, lightweight, smooth, and app-like

[x] Design Style: Bright modern marketplace, clean compact layout, elegant rounded cards

[x] Dense but organized mobile UI with small but very readable typography

[x] Modern startup/fintech/marketplace hybrid style

[x] Smooth animations (60fps), subtle transitions only, avoid heavy effects

## A.2 Color System and Typography

### Primary Color: #7C3AED (Purple)

[x] Gradients: #7C3AED to #3B82F6, #7C3AED to #60A5FA (purple to bright blue)

[x] Used on: buttons, hero sections, active menu, badges, cards, highlights

[x] Bright clean white backgrounds with purple accent dominance

[x] Blue secondary highlights, soft shadows, subtle glassmorphism

### Typography

[x] Small compact text - extremely readable

[x] Modern mobile typography with compact spacing

[x] Marketplace-style UI density, sharp text rendering

## A.3 Layout and Navigation

### Layout Structure

[x] Fixed centered mobile layout (max width: 430px)

[x] Full height mobile app container

[x] Sticky bottom navigation (rounded top corners, floating effect, active gradient indicator)

[x] Sticky top search section

[x] Smooth scroll, lazy loading, skeleton loading components

### Bottom Navigation (5 Tabs)

[x] 1. HOME - hero section, banner carousel, search, categories, featured cards

[x] 2. FEEDS - dummy feed data, card-based timeline, mobile social feed style

[x] 3. PACK - dummy package/product cards, compact marketplace layout

[x] 4. AGEN - dummy agent list, compact cards, contact buttons, status badges

[x] 5. AKUN - Google OAuth login/register or profile dashboard after login

### Header / Top Toolbar

[x] Currency switcher, Language switcher, Notification icon, Search, Compact mobile toolbar

## A.4 Pages Specification

### HOME PAGE

[x] Modern hero section, auto slider/banner carousel, search pack section

[x] Comparison icon section, category shortcuts, highlight gallery

[x] Featured cards, promotional cards, marketplace-style sections

[x] Lazy loading images with shimmer loading

**FEEDS PAGE - Card-based timeline, mobile social feed style**

**PACK PAGE - Package/product cards, compact marketplace layout, modern card design**

**AGEN PAGE - Agent list, compact cards, contact buttons, status badges**

**AKUN PAGE**

[x] Before login: Modern login/register screen, Google Auth only (no manual email/password)

[x] After login: Dashboard profile, profile card, activity summary, menu list, settings, logout

## **A.5 Technical Specification**

[x] Tech Stack: Next.js App Router + Tailwind CSS + shadcn/ui

[x] SEO friendly, PWA ready, Accessibility support

[x] Reusable components: cards, buttons, tabs, modals, sliders, skeleton loaders, image galleries, badges, floating buttons

[x] Modals: bottom sheet modal, popup modal, toaster notifications, animated transitions

[x] Performance: Ultra fast, lazy loading, optimized images, lightweight rendering, mobile low-end device optimized

# PART B - BACKEND: ALFIAN TOUR FULL STACK BLUEPRINT v2.0

Sumber: AlfianTour\_Blueprint\_v2.docx

.NET Core 10 | Web API | EF Core | SQL Server | Clean Architecture

## B.1 Solution Structure (Clean Architecture 4 Projects)

```
TravelApp.sln
src/
  TravelApp.Domain/      # Entities, Enums, Value Objects
  TravelApp.Application/ # DTOs, Interfaces, Services, Validators
  TravelApp.Infrastructure/ # EF Core, Repositories, Migrations
  TravelApp.API/         # Controllers (v1), Middleware, Program.cs
tests/
  TravelApp.Tests/
```

## B.2 NuGet Packages (.NET 10)

Infrastructure: EF Core SqlServer 10.x, Tools 10.x, Design 10.x

Application: AutoMapper 12.x, FluentValidation 11.x, MediatR 12.x, HtmlAgilityPack 1.x, Slugify.Core 3.x

API: Swashbuckle 7.x, API Versioning 8.x, JwtBearer 10.x, Serilog 8.x, ImageSharp 3.x

## B.3 Domain: Enums and BaseEntity

### Enums

RegistrationStatus: Pending, Confirmed, WaitingPayment, Paid, Cancelled, Completed

RoomType: Quad=4, Triple=3, Double=2, Single=1

Gender: Male, Female

AgencyType: HeadOffice, Branch, IndependentAgent

PaymentMethod: BankTransfer, Cash, Other

PaymentStatus: Pending, PartialPaid, FullPaid, Refunded

TicketOwnershipStatus: NotOwned, AlreadyOwned

FeedPostStatus: Draft, Published, Archived

LeadStatus: New, Contacted, Interested, Quoted, Won, Lost, Nurturing

LeadSource: Website, WhatsApp, Instagram, Facebook, Referral, Walk\_In, Other

### BaseEntity (v2.0)

```
public abstract class BaseEntity {  
    public int Id { get; set; }  
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;  
    public DateTime? UpdatedAt { get; set; }  
    public bool IsDeleted { get; set; } = false;  
}
```

## B.4 Core Entities Summary

PackageType (Layanan): Umrah, Haji, Wisata, Umrah Private, Badal Umrah, Badal Haji

Airline: Maskapai + IATA Code (CRUD wajib)

Hotel: Hotel + Star Rating + Distance from Landmark + Budget Range

InsuranceType: Asuransi WAJIB per paket, dinamis (CRUD)

Airport: Bandara asal/tujuan dinamis (CRUD)

PackageLabelTag: Label dinamis (Promo, Best, Premium) dengan custom warna + icon

## B.5 Package Enhanced (v2.0 Features)

[x] Label Tag Dinamis: PackageLabelTagId (FK, nullable)

[x] Harga Coret: OriginalPriceIDR (strikethrough)

[x] Harga Promo: PromoPrice + PromoStartDate + PromoEndDate

[x] Dual Pricing by Ticket: PriceWithTicket / PriceWithoutTicket

[x] Insurance WAJIB: InsuranceTypeId (FK required)

[x] SEO: MetaTitle, MetaDescription, MetaKeywords, OgImageUrl, SchemaOrgJson

[x] Inventory: TotalSeats, RemainingSeats (atomic decrement on confirm)

[x] Relations: Schedules, Facilities, Images, Itineraries, Registrations, Testimonials

## B.6 Departure Engine (v2.0)

## PackageSchedule

[x] FK to Airport: DepartureAirportId + ArrivalAirportId

[x] Dates: DepartureDateUtc, ReturnDateUtc, DurationDays

[x] Stay Breakdown: e.g. '3 malam Madinah, 5 malam Makkah'

[x] Custom Itinerary per Schedule: HasCustomItinerary flag

[x] Price Override: PriceOverrideIDR for specific schedules

[x] Seat per Schedule: ScheduleRemainingSeats

## ScheduleItinerary

[x] Per schedule can have custom itinerary: DayNumber, DayTitle, Description, Location, MealInfo, HotelInfo

## B.7 Feeds / Social Media Platform (v2.0)

## Access Control

[x] PUBLIC: View Posts (List + Detail), Hashtag Search, Share

[x] USER (Login): Toggle Like, Comment, Delete own comment

[x] Admin/Editor/ContentManager: Create, Edit, Delete Posts

## Entities

[x] FeedPost: Title, Slug, Content (HTML), CoverImageUrl, YoutubeVideoUrl, ImageUrls (JSON), ViewCount, LikeCount (denormalized), CommentCount (denormalized), Status

[x] FeedPostLike: Composite PK (FeedPostId + UserId)

[x] FeedComment: FeedPostId, UserId, Content, ParentCommentId (reply support)

[x] Hashtag + FeedPostHashtag: M:N relationship, UsageCount

## Feeds API Endpoints

|        |                             |                                |
|--------|-----------------------------|--------------------------------|
| GET    | /api/v1/feeds               | # Public list (pagination)     |
| GET    | /api/v1/feeds/{slug}        | # Detail by slug               |
| GET    | /api/v1/feeds/hashtag/{tag} | # Posts by hashtag             |
| GET    | /api/v1/feeds/search?q=     | # Full-text search             |
| POST   | /api/v1/feeds               | # Create (Admin/Editor)        |
| PUT    | /api/v1/feeds/{id}          | # Edit (Admin/Owner)           |
| DELETE | /api/v1/feeds/{id}          | # Soft delete (Admin/Owner)    |
| POST   | /api/v1/feeds/{id}/like     | # Toggle like (User)           |
| POST   | /api/v1/feeds/{id}/comments | # Add comment (User)           |
| DELETE | /api/v1/feeds/comments/{id} | # Delete comment (Owner/Admin) |

## B.8 Leads Management (CRM Pipeline)

[x] Lead Entity: FullName, PhoneNumber, Email, City, LeadStatus, LeadSource, InterestedPackageId, EstimatedPax, BudgetPerPaxIDR, AssignedToUserId, NextFollowUpAtUtc, ConvertedRegistrationId

[x] LeadActivity: ActivityType (Call/WhatsApp/Email/Meeting), Notes, StatusAfter

## B.9 Matrix / KPI Dashboard

[x] Revenue KPI: Total Revenue, Total Bookings, AvgOrderValue per period

[x] Package Performance: Top packages ranking, occupancy rate

[x] Leads Funnel: Conversion rate through pipeline stages

[x] Revenue by Agency: Total revenue per agency branch

[x] Feeds Engagement: Total Posts, Views, Likes, Comments, Top Posts

## B.10 All DbSet in ApplicationDbContext (v2.0)

```
// Core Business
DbSet<PackageType>, DbSet<Package>, DbSet<PackageSchedule>, DbSet<ScheduleItinerary>
DbSet<PackageImage>, DbSet<PackageFacility>, DbSet<PackageItinerary>
DbSet<PackageLabelTag>, DbSet<InsuranceType>, DbSet<Airport>, DbSet<Airline>, DbSet<Hotel>
DbSet<Agency>, DbSet<Registration>, DbSet<RegistrationParticipant>, DbSet<Payment>
DbSet<UmrahPrivateRequest>, DbSet<Testimonial>

// Feeds
DbSet<FeedPost>, DbSet<FeedPostLike>, DbSet<FeedComment>, DbSet<Hashtag>, DbSet<FeedPostHashtag>

// Leads
DbSet<Lead>, DbSet<LeadActivity>

// Blog
DbSet<BlogPost>, DbSet<BlogCategory>, DbSet<BlogPostCategory>
DbSet<AppSetting>
```

## B.11 Seeder Default Data

[x] InsuranceType: Allianz, Takaful, Zurich

[x] PackageLabelTag: Regular, Promo, Best, Premium, New

[x] Airport: CGK, SUB, BDO, JOG, UPG, MED (Madinah), JED (Jeddah)

[x] PackageType: Umrah, Haji, Wisata, Badal Haji, Badal Umrah, Umrah Private

[x] AppSettings: site\_name='Alfian Tour', default\_currency='IDR'

## B.12 Timezone, SEO and Access Control

### Timezone Rules

[x] Database: ALL DateTime stored as UTC

[x] Display: WIB (Asia/Jakarta, UTC+7) at API Response / DTO layer

[x] Input: yyyy-MM-dd | Output: 'Senin, 13 Mei 2026' (id-ID)

## Role-Based Access Control

[x] Admin: Full access - all CRUD, payment confirm, user management

[x] Manager: Matrix/KPI, leads, reports (no user deletion)

[x] Editor/ContentManager: FeedPost and BlogPost CRUD (no finance)

[x] Staff/Sales: Leads CRUD, activity logging, registration list

[x] User (Registered): Like, Comment, Register packages, check status

[x] Public (Anonymous): View posts/packages/blogs, submit lead/request forms

# PART C - MASTER DATA ARCHITECTURE v3.0

Sumber: travelapp\_master\_data\_architecture\_prompt\_v\_3.md

Platform: .NET 10 | ASP.NET Core Web API | EF Core | SQL Server | Clean Architecture

## C.1 Global Rules (CRITICAL)

### Currency Rules

[x] ALL prices stored in USD in database - NEVER use PriceIDR fields

[x] Use Price or BasePrice (decimal 18,2)

[x] Currency conversion at Frontend / API Response Layer / CurrencyService

[x] Database ALWAYS stores USD; display: IDR, USD, SAR, MYR, SGD, EUR, GBP

### Audit Rules (MANDATORY)

[x] Every entity: Audit Trail, Soft Delete, Active/Inactive, Restore, User Tracking

[x] BaseAuditableEntity: CreatedAtUtc, UserId, UpdatedAtUtc, UpdatedByUserId, DeletedAtUtc, DeletedByUserId, IsDeleted, IsActive

[x] All FK to User use string UserId (connected to [Identity].[Users].[Id])

### BaseAuditableEntity (v3.0 - Enhanced)

```
public abstract class BaseAuditableEntity {  
    public int Id { get; set; }  
    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;  
    public string UserId { get; set; } = string.Empty;  
    public DateTime? UpdatedAtUtc { get; set; }  
    public string? UpdatedByUserId { get; set; }  
    public DateTime? DeletedAtUtc { get; set; }  
    public string? DeletedByUserId { get; set; }  
    public bool IsDeleted { get; set; } = false;  
    public bool IsActive { get; set; } = true;  
}
```

## C.2 15 Master Data Entities

1. PackageType - Master layanan: Umrah, Haji, Wisata, Umrah Private, Badal Umrah, Badal Haji
2. PackageCategory - Kategori/class: Regular, VIP, Executive, Backpacker, Corporate, Family, Group
3. Airline - Maskapai + IATA Code + Logo + IsInternational
4. Airport - Bandara + IATA Code + City + Country
5. Hotel - Hotel + City + Country + StarRating + DistanceFromLandmark + DistanceKm + PriceRange
6. InsuranceType - Asuransi + Provider + PremiumPerPax
7. PackageLabelTag - Label dinamis + ColorHex + TextColorHex + IconKey
8. RoomTypeMaster - Jenis kamar: Quad(4), Triple(3), Double(2), Single(1) + Capacity
9. FacilityMaster - Fasilitas + IconKey + IsHighlight
10. RequirementMaster - Persyaratan + IsMandatory
11. TermsTemplate - Template syarat dan ketentuan

12. DurationType - Tipe durasi: TotalDays + TotalNights

13. DepartureSeason - Musim keberangkatan + StartDate + EndDate

14. CurrencyMaster - Mata uang: Code + Symbol + ExchangeRate + IsBaseCurrency

15. MediaAsset - Upload file: FileName + Url + MimeType + FileSize + AltText

### C.3 Application and Infrastructure Layers

[x] FluentValidation: Name wajib, Slug unique, Currency code unique, IATA unique, ExchangeRate > 0, Capacity > 0

[x] Response: { success, message, data }

[x] Pagination: { items[], page, pageSize, totalCount, totalPages }

[x] Global Query Filter: builder.HasQueryFilter(x => !x.IsDeleted)

[x] Soft Delete: IsDeleted=true, DeletedAtUtc=now, DeletedByUserId=currentUserId

## C.4 API v1 Master Data Endpoints Pattern

|        |                                      |                                     |
|--------|--------------------------------------|-------------------------------------|
| GET    | /api/v1/master/{entity}              | # List + Pagination + Search + Sort |
| GET    | /api/v1/master/{entity}/{id}         | # Detail                            |
| POST   | /api/v1/master/{entity}              | # Create                            |
| PUT    | /api/v1/master/{entity}/{id}         | # Update                            |
| DELETE | /api/v1/master/{entity}/{id}         | # Soft Delete                       |
| POST   | /api/v1/master/{entity}/{id}/restore | # Restore                           |

Applies to all 15 entities: package-types, package-categories, airlines, airports, hotels, insurance-types, package-label-tags, room-types, facility-masters, requirement-masters, terms-templates, duration-types, departure-seasons, currencies, media-assets

## C.5 Seeder Data

Currency: USD (Base), IDR, SAR, MYR, SGD, EUR, GBP

PackageType: Umrah, Haji, Wisata, Umrah Private

RoomType: Quad, Triple, Double, Single

LabelTag: Promo, Best, Premium, Limited, New

# PART D - FINANCIAL AND COSTING ARCHITECTURE

Sumber: travelapp\_financial\_architecture\_blueprint.md

Tujuan: Owner dapat melihat keuntungan, omzet, modal, fee, gaji staff, operasional, profit per paket, cashflow, hutang/piutang

## D.1 4-Layer Financial Architecture

Layer 1: Master Cost - Harga dasar vendor (Hotel, Airline, Visa, Facility)

Layer 2: Package Costing - Biaya paket dan breakdown modal

Layer 3: Transaction - Booking dan pembayaran

Layer 4: Financial Ledger - Cashflow dan laporan keuangan

## D.2 Master Cost Data (4 Entities)

## 1. HotelCost

Harga modal hotel per malam: CostPerNight, SellPricePerNight, CurrencyCode=USD, EffectiveDateUtc, EndDateUtc

## 2. AirlineTicketCost

Harga modal tiket: TicketCost, SellPrice, CurrencyCode=USD, IsRoundTrip, DepartureAirportId, ArrivalAirportId

## 3. ServiceCost (Visa/Passport/Document)

Biaya dokumen: Name, Category (Visa/Passport/Handling), CostPrice, SellPrice, IsPerPax

## 4. FacilityCost

Modal fasilitas (Bus, Tour Guide, Handling, Zamzam, Makan, Laundry, Kereta Cepat): CostPrice, SellPrice, VendorName

## D.3 Package Costing System (MOST IMPORTANT)

## 5. PackageCosting

Breakdown modal paket: TotalCost, TotalSellPrice, EstimatedProfit, ProfitMarginPercent, CurrencyCode=USD

Relation: PackageCosting (1) -> PackageCostingItem (N)

## 6. PackageCostingItem

Detail: Category (Hotel/Tiket/Visa/Handling), ItemName, CostPrice, SellPrice, Quantity, TotalCost, TotalSell, ProfitAmount, VendorName

## D.4 Owner Fee and Commission System

## 7. ProfitSharingConfig

Pembagian profit: OwnerPercent (40%), MarketingPercent (20%), OperationalPercent (20%), ReservePercent (20%), IsDefault

## 8. EmployeeSalary

Gaji staff: BaseSalary, BonusAmount, CommissionAmount, DeductionAmount, FinalSalary, SalaryMonthUtc

## D.5 Financial Accounting

### 9. FinancialAccount (Chart of Accounts)

AccountCode, AccountName, AccountType (Asset/Liability/Revenue/Expense), ParentAccountCode, SortOrder

### 10. FinancialTransaction (Cashflow)

All money in/out: TransactionCode, TransactionType (Income/Expense), FinancialAccountId, Amount, CurrencyCode=USD, ReferenceNumber, RegistrationId, PackageId, VendorName

### 11. OperationalExpense

Biaya kantor: ExpenseName, Amount, ExpenseDateUtc, VendorName (listrik, internet, iklan, gaji, sewa, operasional mobil)

## D.6 Insurance Master and Profit System

IMPORTANT: JANGAN hanya simpan 'Asuransi = 35 USD'. Insurance system harus flexible dan scalable karena variasi umur, benefit, provider, plan, fee, markup, komisi. Ini hidden profit terbesar di bisnis travel.

## 12. InsuranceProvider

Master perusahaan: Name, LogoUrl, Description, WebsiteUrl, ContactPerson, PhoneNumber (Allianz, Takaful, Zurich, AXA, Prudential)

## 13. InsuranceProduct

Produk per provider: Name, Slug, Description, CurrencyCode=USD, IsRecommended (Umrah Basic, Umrah Premium, Umrah VIP, Haji Protection)

## 14. InsuranceAgeRange

Harga berdasarkan umur: MinAge, MaxAge, CostPrice, SellPrice, FeeAmount (fee travel). Contoh: Umur 1-40 = 20/30 USD, Umur 41-60 = 35/50 USD

## 15. InsuranceBenefit

Coverage: BenefitName, Description, CoverageAmount, IsHighlight. Contoh: Pemulangan Jenazah (,000), Rawat Inap (,000), Kehilangan Bagasi, Covid Protection

## D.7 Insurance PDF AI Import Engine

Flow: PDF Insurance -> AI OCR/PDF Reader -> Extract JSON -> Validate -> Create InsuranceProduct -> Create InsuranceAgeRange -> Create InsuranceBenefit

Supports: PDF, Excel, CSV, Image Scan

Insurance Profit Example: Provider cost 25 USD, Sell to jamaah 40 USD, Fee travel = 15 USD. For 100 jamaah = 1,500 USD profit.

## D.8 Owner Dashboard and Financial Reports

### Dashboard KPIs

[x] Total Revenue, Total Profit, Total Expense, Operational Cost

[x] Owner Profit, Employee Salary, Monthly Revenue

[x] Top Selling Package, Profit Per Package, Profit Per Departure

[x] Outstanding Payment, Total Booking, Total Jamaah

### Financial Reports Available

[x] Cashflow Report, Profit and Loss, Revenue Report, Expense Report

[x] Salary Report, Vendor Payment Report, Profit Margin Report

[x] Package Profitability, Lead Conversion Revenue

### Priority Build Order

[x] PRIORITY 1: PackageCosting, PackageCostingItem, FinancialTransaction, OperationalExpense, FinancialAccount

[x] PRIORITY 2: EmployeeSalary, Commission, ProfitSharingConfig

[x] PRIORITY 3: Full Accounting, Balance Sheet, General Ledger, Journal Entry

# PART E - SCALABILITY AUDIT AND BOTTLENECK PREVENTION (NEW)

Audit ini mengidentifikasi potensi bottleneck dan menyediakan solusi skalabilitas jangka panjang.

## E.1 Database Scalability Analysis

### E.1.1 Current Architecture Review

Current: SQL Server monolitik + EF Core. Adequate for small-medium scale (<100K users, <1M records). For scale-up preparation, several areas need reinforcement.

### E.1.2 Indexing Strategy (CRITICAL)

Without proper indexing, queries become the main bottleneck as data grows.

[x] MANDATORY: Composite index on (IsDeleted, IsActive) for all entities

[x] MANDATORY: Index on FK columns: PackageId, UserId, AgencyId, RegistrationId

[x] MANDATORY: Full-Text Index on: FeedPost.Content, FeedPost.Title, Package.Title, Package.Description

[x] MANDATORY: Index on date columns: CreatedAtUtc, DepartureDateUtc, PublishedAtUtc

[x] MANDATORY: Unique index on Slug fields (Package, FeedPost, BlogPost)

[x] RECOMMENDED: Filtered index WHERE IsDeleted = 0 (reduces index size)

```
CREATE INDEX IX_Packages_Active ON Packages (IsDeleted, IsActive) WHERE IsDeleted = 0;  
CREATE FULLTEXT CATALOG TravelAppCatalog;  
CREATE FULLTEXT INDEX ON FeedPosts (Title, Content) KEY INDEX PK_FeedPosts ON TravelAppCatalog;
```

### E.1.3 Query Optimization Patterns

[x] USE AsNoTracking() for ALL read queries - reduces EF Core memory overhead

[x] USE Select() projection (DTO mapping) - never load entire entity graph

[x] USE keyset/cursor-based pagination for feeds (NOT OFFSET-based, slow on large pages)

[x] AVOID N+1 queries - use .Include() selectively or split queries

[x] LIMIT Include depth to max 2 levels

[x] USE compiled queries for frequently called queries

#### **E.1.4 Read/Write Splitting (CQRS Lite)**

[x] PRIMARY DB: Handle ALL write operations (CREATE, UPDATE, DELETE)

[x] READ REPLICA: Handle ALL read operations (GET, LIST, SEARCH)

[x] EF Core supports this via separate connection strings

[x] Replica latency (1-2 sec) acceptable for non-critical data (feeds, blog)

#### **E.1.5 Data Archiving Strategy**

[x] Fast-growing tables: FinancialTransaction, LeadActivity, FeedPostLike, FeedComment

[x] ARCHIVE: Records older than 2 years moved to separate archive database

[x] PARTITIONING: Partition by Year/Month on FinancialTransaction, Registration

[x] Use SQL Server Partitioning for transaction tables (Partition Function by Date)

## E.2 API Performance and Caching Strategy

### E.2.1 Multi-Layer Caching Architecture

[Client] -> [CDN Cache (Static Assets)] -> [API Gateway] -> [Redis Cache] -> [API Server] -> [SQL Server]

### E.2.2 Redis Cache Strategy

[x] Cache-Aside Pattern: Check cache -> if miss, query DB -> update cache

[x] TTL Strategy: Master data = 24h, Package listings = 1h, Feeds = 15min, Currency rates = 24h, Matrix/KPI = 30min

[x] Cache Invalidation: Redis Pub/Sub for invalidation on data changes

[x] Cache Key Pattern: 'travelapp:{entity}:{id}', 'travelapp:{entity}:list:{page}:{filters}'

### E.2.3 Response Compression and CDN

[x] GZIP/Brotli compression for all API responses

[x] Static assets via CDN (CloudFront/Cloudflare)

[x] Image optimization: WebP format, responsive sizing, lazy loading

[x] API Response Caching: Cache-Control headers for public endpoints

#### **E.2.4 Rate Limiting and Throttling**

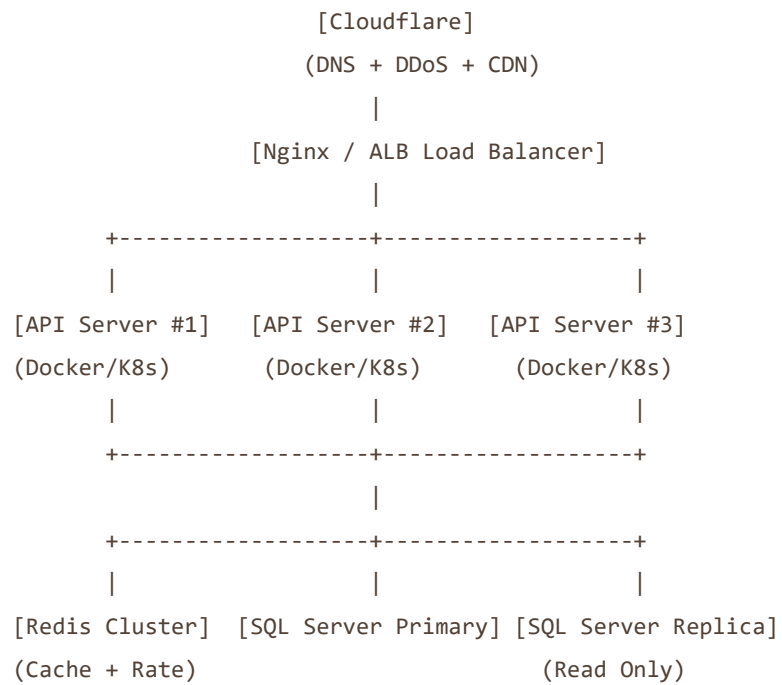
[x] Rate limits: Public feeds = 100 req/min, Auth endpoints = 30 req/min

[x] Use ASP.NET Core Rate Limiting middleware

[x] Distributed rate limiting via Redis for multi-server deployment

### **E.3 Infrastructure and Deployment Architecture**

#### **E.3.1 Recommended Production Architecture**



### E.3.2 Containerization (Docker + K8s)

[x] Dockerize each service: API, Background Worker (Hangfire), Migration Runner

[x] Kubernetes HPA based on CPU/Memory utilization

[x] Health Checks: /healthz endpoint for readiness and liveness probes

[x] Graceful shutdown: 30 second timeout for in-flight requests

### **E.3.3 Background Jobs and Message Queue**

[x] Hangfire/Quartz.NET for: PDF Insurance Import, Email/SMS notifications, Report generation, Data archiving, Cache warming

[x] RabbitMQ/Azure Service Bus for: Event-driven inter-service communication, Async notification dispatch, Audit log streaming

## **E.4 Anti-Bottleneck Design Patterns**

### **E.4.1 Denormalization for Read Performance**

[x] FeedPost.LikeCount/CommentCount: Updated via atomic increment/decrement

[x] Package.RemainingSeats: Updated via atomic operation on booking confirm/cancel

[x] Hashtag.UsageCount: Updated when posts are created/deleted

[x] Use database transactions + optimistic concurrency (RowVersion) for race condition prevention

### E.4.2 Connection Pool Management

[x] EF Core Max Pool Size = 200 (default 100)

[x] Connection timeout: 30 seconds (default 15)

[x] Command timeout: 60 seconds for reporting queries

[x] MARS = false (avoid overhead)

[x] Limit parallel queries to max 3 concurrent

### E.4.3 Bulk Operations Optimization

[x] Use SqlBulkCopy for bulk insert (NOT EF Core AddRange)

[x] Batch size: 1000-5000 records per batch

[x] Disable change tracking during bulk operations

[x] Use stored procedures for heavy aggregation (Matrix/KPI)

#### **E.4.4 Feeds Performance Pattern (Critical)**

[x] Use Cursor-based Pagination (Keyset) - NOT OFFSET

Query: WHERE PublishedAtUtc < @cursor ORDER BY PublishedAtUtc DESC FETCH NEXT 20 ROWS ONLY

[x] PRE-COMPUTE trending feeds at write time (fan-out on write)

[x] Use materialized views for aggregation

[x] Separate read model: FeedPostListView (flattened, indexed, optimized)

### **E.5 Monitoring, Observability and Alerting**

#### **E.5.1 Logging and Tracing**

[x] Structured Logging: Serilog -> Elasticsearch / Seq / Application Insights

[x] Distributed Tracing: OpenTelemetry with trace context propagation

[x] Correlation ID: Every request has CorrelationId propagated to downstream calls

### **E.5.2 Metrics and Dashboards**

[x] Infrastructure: CPU, Memory, Disk I/O, Network (Prometheus + Grafana)

[x] Application: Request rate, Latency (p50/p95/p99), Error rate, Throughput

[x] Database: Connection pool, Query execution time, Deadlocks, Index usage

[x] Business: Booking rate, Conversion rate, Revenue per minute

### **E.5.3 Alerting Rules**

[x] CRITICAL: API error rate above 5% -- immediate alert

[x] CRITICAL: Database connection pool exhausted -- immediate alert

[x] WARNING: API latency p95 above 2000ms -- alert

[x] WARNING: Redis cache miss rate above 30% -- alert

[x] INFO: Disk usage above 80% -- alert

[x] INFO: Remaining seats less than 5 for popular packages -- business alert

## E.6 Data Consistency and Concurrency Control

### E.6.1 Optimistic Concurrency (RowVersion)

```
// BaseEntity addition:
public byte[] RowVersion { get; set; } = Array.Empty<byte>();
// EF Config:
builder.Property(e => e.RowVersion).IsRowVersion();
// Handle conflict:
try { await _context.SaveChangesAsync(); }
catch (DbUpdateConcurrencyException) { /* Retry or return conflict */ }
```

### E.6.2 Critical Concurrency Scenarios

### 1. SEAT BOOKING: 2 users booking last seat simultaneously

[x] Use transaction + RowVersion: BeginTransaction -> Check RemainingSeats > 0 -> Decrement -> SaveChanges -> Commit

[x] On failure: Retry max 3x with exponential backoff

### 2. PAYMENT PROCESSING: Double payment prevention

[x] Use Idempotency Key on payment requests

[x] Check idempotency key -> If already processed, return existing result

### 3. LIKE/COMMENT COUNTER: Race condition on denormalized counter

[x] Use database-level atomic increment: UPDATE FeedPosts SET LikeCount = LikeCount + 1 WHERE Id = @id

## E.6.3 Saga Pattern for Distributed Transactions

[x] For future multi-service operations: Booking = CreateRegistration -> DeductSeat -> CreatePayment -> SendNotification

[x] Each step has compensating transaction for rollback

[x] Outbox Pattern for reliable message publishing

## E.7 Future-Proof Architecture Recommendations

### E.7.1 Microservice Extraction Path

[x] PHASE 1: Extract Financial Service (standalone, dedicated DB)

[x] PHASE 2: Extract Notification Service (Email, SMS, WhatsApp, Push)

[x] PHASE 3: Extract Feeds/Content Service (high read, low write)

[x] PHASE 4: Extract CRM/Leads Service

### E.7.2 Database Scaling Path

[x] CURRENT: Single SQL Server (0-500K users)

[x] PHASE 1: Add Read Replica + Redis Cache (500K-2M users)

[x] PHASE 2: Partitioning + Sharding by Tenant/Agency (2M-10M users)

[x] PHASE 3: Event Sourcing + CQRS full (10M+ users)

[x] PHASE 4: Multi-region with Geo-replication

### **E.7.3 Technology Watch List**

[x] GraphQL for flexible mobile API (reduce over-fetching)

[x] gRPC for inter-service communication (high performance)

[x] Event Store DB for event sourcing

[x] Elasticsearch for full-text search (replace SQL FTS)

[x] Azure SQL Hyperscale / AWS Aurora for auto-scaling database

# PART F - CONSOLIDATED DATABASE SCHEMA (ALL TABLES)

Total: 40+ tables across all 4 source documents

## F.1 Core Business Tables (18 tables)

PackageTypes - Jenis layanan (Umrah, Haji, Wisata)

Packages - Produk paket utama (Enhanced v2) | FK: PackageTypeId, AirlineId, HotelMakkahId, HotelMadinahId, InsuranceTypeId, PackageLabelTagId

PackageSchedules - Jadwal keberangkatan | FK: PackageId, DepartureAirportId, ArrivalAirportId

ScheduleItineraries - Itinerary per jadwal | FK: PackageScheduleId

PackageImages - Foto per paket | FK: PackageId

PackageFacilities - Fasilitas paket | FK: PackageId

PackageItineraries - Itinerary global | FK: PackageId

Airlines - Maskapai + IATA

Airports - Bandara dinamis

Hotels - Hotel + rating + jarak

InsuranceTypes - Asuransi wajib

PackageLabelTags - Label dinamis (Promo, Best, dll)

Agencies - Cabang dan agen | FK: ParentAgencyId (self-ref)

Registrations - Booking | FK: PackageId, ScheduleId, AgencyId

RegistrationParticipants - Data peserta | FK: RegistrationId

Payments - Pembayaran | FK: RegistrationId

UmrahPrivateRequests - Request umrah private | FK: AgencyId

Testimonials - Testimoni jamaah | FK: Packageld

## F.2 Master Data Tables (9 tables from v3.0)

PackageCategories, RoomTypeMasters, FacilityMasters, RequirementMasters, TermsTemplates, DurationTypes, DepartureSeasons, CurrencyMasters, MediaAssets

## F.3 Financial Tables (11 tables)

HotelCosts | FK: HotelId

AirlineTicketCosts | FK: AirlineId, DepartureAirportId, ArrivalAirportId

ServiceCosts - Biaya dokumen/visa

FacilityCosts | FK: FacilityMasterId

PackageCostings - Breakdown biaya paket | FK: Packageld

PackageCostingItems - Detail item | FK: PackageCostingId

ProfitSharingConfigs - Konfigurasi bagi hasil

EmployeeSalaries - Gaji staff | FK: EmployeeUserId

FinancialAccounts - Chart of Accounts | FK: ParentAccountCode (self-ref)

FinancialTransactions - Cashflow | FK: FinancialAccountId, RegistrationId, PackageId

OperationalExpenses - Biaya operasional

## **F.4 Insurance System Tables (4 tables)**

InsuranceProviders - Perusahaan asuransi

InsuranceProducts - Produk per provider | FK: InsuranceProviderId

InsuranceAgeRanges - Harga by umur | FK: InsuranceProductId

InsuranceBenefits - Coverage | FK: InsuranceProductId

## F.5 Feeds / Social Media (5 tables)

FeedPosts - Post feeds publik | FK: UserId

FeedPostLikes - Like per user (Composite PK: FeedPostId + UserId)

FeedComments - Komentar + reply (self-ref via ParentCommentId) | FK: FeedPostId, UserId

Hashtags - Master hashtag

FeedPostHashtags - Join M:N (Composite PK: FeedPostId + HashtagId)

## F.6 Leads / CRM (2 tables)

Leads - Pipeline prospek | FK: InterestedPackageTypeId, InterestedPackageId, AssignedToUserId, AgencyId, ConvertedRegistrationId

LeadActivities - Riwayat kontak | FK: LeadId, UserId

## F.7 Blog and Settings (4 tables)

BlogPosts - Artikel + SEO | FK: UserId

BlogCategories - Kategori artikel

BlogPostCategories - Join M:N (Composite PK)

AppSettings - Key-Value config global

# PART G - COMPLETE API ENDPOINT REFERENCE

## G.1 Public Endpoints (No Auth Required)

GET /api/v1/packages List paket (filter: typeSlug, labelTag, search)

GET /api/v1/packages/{slug} Detail paket by slug

GET /api/v1/packages/featured Paket unggulan

GET /api/v1/packages/{id}/schedules List jadwal keberangkatan

GET /api/v1/airlines List maskapai

GET /api/v1/airports List bandara

GET /api/v1/insurance-types List asuransi

GET /api/v1/package-label-tags List label (Promo, Best, dll)

|      |                               |                               |
|------|-------------------------------|-------------------------------|
| GET  | /api/v1/feeds                 | List post publik (pagination) |
| GET  | /api/v1/feeds/{slug}          | Detail post by slug           |
| GET  | /api/v1/feeds/hashtag/{tag}   | Post by hashtag               |
| GET  | /api/v1/feeds/search?q=       | Pencarian full-text           |
| GET  | /api/v1/currency/usd-idr      | Kurs USD-IDR (cache 24h)      |
| POST | /api/v1/registrations         | Daftar paket (optional auth)  |
| POST | /api/v1/payments/submit       | Upload bukti bayar            |
| POST | /api/v1/umrah-private/request | Request Umrah Private         |
| POST | /api/v1/leads                 | Submit lead (public form)     |

## G.2 Authenticated User Endpoints

|        |                             |                           |
|--------|-----------------------------|---------------------------|
| POST   | /api/v1/feeds/{id}/like     | Toggle like post          |
| POST   | /api/v1/feeds/{id}/comments | Tambah komentar           |
| DELETE | /api/v1/feeds/comments/{id} | Hapus komentar sendiri    |
| GET    | /api/v1/registrations/my    | List booking user sendiri |

### G.3 Admin/Staff Endpoints

|        |                                 |                           |
|--------|---------------------------------|---------------------------|
| POST   | /api/v1/packages                | Tambah paket (Admin)      |
| PUT    | /api/v1/packages/{id}           | Edit paket (Admin)        |
| DELETE | /api/v1/packages/{id}           | Soft delete paket (Admin) |
| POST   | /api/v1/packages/{id}/schedules | Tambah jadwal (Admin)     |
| PUT    | /api/v1/schedules/{id}          | Edit jadwal (Admin)       |

|        |                                    |                                   |
|--------|------------------------------------|-----------------------------------|
| POST   | /api/v1/feeds                      | Buat post (Admin/Editor)          |
| PUT    | /api/v1/feeds/{id}                 | Edit post (Admin/Owner)           |
| DELETE | /api/v1/feeds/{id}                 | Hapus post (Admin/Owner)          |
| POST   | /api/v1/registrations/{id}/confirm | Konfirmasi booking (kurangi seat) |
| POST   | /api/v1/payments/confirm           | Konfirmasi pembayaran             |
| GET    | /api/v1/leads                      | List leads (filterable)           |
| PUT    | /api/v1/leads/{id}                 | Update status lead                |
| POST   | /api/v1/leads/{id}/activities      | Catat aktivitas lead              |

## G.4 Manager/KPI Endpoints

|     |                        |                           |
|-----|------------------------|---------------------------|
| GET | /api/v1/matrix/revenue | KPI pendapatan (from, to) |
|-----|------------------------|---------------------------|

GET /api/v1/matrix/packages-performance Kinerja paket (limit)

GET /api/v1/matrix/leads-funnel Funnel leads

GET /api/v1/matrix/revenue-by-agency Revenue per agency

GET /api/v1/matrix/feeds-engagement Engagement feeds

## G.5 Master Data CRUD Endpoints (15 entities)

All via /api/v1/master/{entity}: GET LIST | GET/{id} | POST | PUT/{id} | DELETE/{id} | POST/{id}/restore

Entities: package-types, package-categories, airlines, airports, hotels, insurance-types, package-label-tags, room-types, facility-masters, requirement-masters, terms-templates, duration-types, departure-seasons, currencies, media-assets

# PART H - BUILD ROADMAP AND IMPLEMENTATION PRIORITIES

## H.1 Phase 1: Foundation (Month 1-2)

[x] Solution setup (.NET 10 + Next.js 14)

[x] BaseAuditableEntity + Audit Trail + Soft Delete System

[x] Identity integration (Google OAuth only)

[x] 15 Master Data entities + CRUD API + Seeder

[x] Core Business entities (Package, Airline, Hotel, Airport, Insurance, LabelTag)

[x] Frontend: Layout + Bottom Navigation + 5 Page Shell

[x] Frontend: Google Auth integration

[x] EF Core configurations + Initial Migration

[x] Swagger documentation

## H.2 Phase 2: Core Business (Month 3-4)

[x] Package Enhanced (labels, promo pricing, ticket types)

[x] Departure Engine (schedules, custom itineraries)

[x] Registration + Participant + Payment flow

[x] Agency management

[x] Frontend: Home page (hero, slider, search, categories)

[x] Frontend: Pack page (package cards, filter, detail)

[x] Frontend: Agen page (agent list, cards)

[x] SEO implementation (meta tags, schema.org)

### H.3 Phase 3: Engagement and CRM (Month 5-6)

[x] Feeds/Social Media (FeedPost, Like, Comment, Hashtag)

[x] Leads Management (Lead, LeadActivity, Pipeline)

[x] Blog system

[x] Frontend: Feeds page (timeline, like, comment)

[x] Frontend: Akun page (profile, history, settings)

[x] Matrix/KPI Dashboard

[x] Redis caching implementation

### H.4 Phase 4: Financial System (Month 7-8)

[x] Master Cost Data (HotelCost, AirlineTicketCost, ServiceCost, FacilityCost)

[x] Package Costing System (PackageCosting + CostingItems)

[x] Profit Sharing Configuration

[x] Financial Accounts (Chart of Accounts)

[x] Financial Transactions (Cashflow)

[x] Operational Expenses

[x] Employee Salary system

[x] Insurance Master and Profit System (Provider, Product, AgeRange, Benefit)

[x] Owner Dashboard (Revenue, Profit, Expense, KPI)

## **H.5 Phase 5: Advanced and Scale (Month 9-12)**

[x] Insurance PDF AI Import Engine

[x] Read Replica + CQRS Lite implementation

[x] Background job system (Hangfire)

[x] Full-text search optimization (Elasticsearch)

[x] Rate limiting + DDoS protection

[x] Docker + Kubernetes deployment

[x] Monitoring stack (Prometheus + Grafana)

[x] Load testing and performance optimization

[x] Multi-currency frontend conversion

[x] PWA + offline support